

# TrueID AI - Backend Handover Document

Spring Boot + MySQL backend: setup history, complete code, how it works, and the API contract. Status as of 3 October 2026. Written by Harshita (backend) for Khushi (frontend) and Karnika (AI service).

## 0. Read this first (instructions for any AI assistant or teammate continuing this work)

This document is the single source of truth for the current state of the TrueID AI backend. If you paste it into an AI chat, ask the assistant to follow these points:

- The backend is **Spring Boot (Java) + MySQL**, package `com.trueid`. It is NOT Node/Express/MongoDB. Do not redesign it.
- The architecture is fixed: React (Vite) frontend -> Spring Boot backend on port 8080 -> FastAPI AI service on port 8000. The backend also talks to MySQL (database `trueid_db`).
- The endpoints, request formats and response JSON in section 8 are the contract. Do not change them without telling Harshita, because the frontend will be built against them.
- The AI result is currently a **mock** inside the backend. The real AI service by Karnika will replace it later. The response format must stay the same (section 10).
- Give code one file at a time, with the exact file path, and keep Java file names identical to the class names (case-sensitive).

### Ready-to-paste prompt:

I am continuing the TrueID AI project. The attached document describes the finished backend (Spring Boot + MySQL), its code, and the API contract. Treat it as the current state. Do not redesign anything. My teammates are Khushi (React frontend) and Karnika (Python FastAPI AI service). Help me with the next step I ask about, and keep all endpoints and JSON formats exactly as documented.

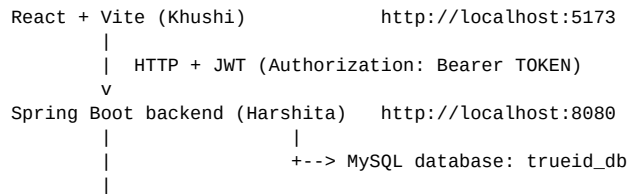
## 1. Project overview

TrueID AI is an identity screening web app. A user signs in, chooses a document type (Aadhaar, Passport, Driving Licence, Voter ID), uploads or scans the document, and receives a screening result (status, risk, confidence, risk indicators). The app also shows history and reports.

### Team and responsibilities

| Person   | Part                                                                            | Technology                      |
|----------|---------------------------------------------------------------------------------|---------------------------------|
| Khushi   | Frontend: all pages, navigation, api.js integration                             | React + Vite (port 5173)        |
| Harshita | Backend: auth, screening records, file upload, history, reports, calling the AI | Spring Boot + MySQL (port 8080) |
| Karnika  | AI service: document type, OCR, validation, tampering, face match, risk score   | Python + FastAPI (port 8000)    |

### Architecture



--> AI service (Karnika) FastAPI, http://localhost:8000 (planned: POST /ai/analyze)  
Currently replaced by a MOCK inside the backend (AIService.java)

## Agreed repository structure

```
TrueID-AI/  
  frontend/      React + Vite (Khushi)  
  backend/       Spring Boot + Java (Harshita)  
  ai-service/    Python + FastAPI (Karnika)  
  database/      schema.sql, seed_data.sql (optional; tables are auto-created by Hibernate)  
  config/        .env.example, README.md  
  docs/          API_DOCUMENTATION.md, AI_DOCUMENTATION.md, SYSTEM_ARCHITECTURE.md
```

## 2. Current status

| Item                                                                  | Status                         |
|-----------------------------------------------------------------------|--------------------------------|
| Project setup (Spring Boot, MySQL connection)                         | Done and tested                |
| Health check: GET /api/health                                         | Done and tested                |
| Signup, login with BCrypt password hashing and JWT                    | Done and tested                |
| JWT filter, protected routes, CORS for the frontend                   | Done and tested                |
| GET and PUT /api/auth/profile                                         | Done and tested                |
| POST /api/screening (file upload, saves result in MySQL)              | Done and tested (AI is a MOCK) |
| GET /api/screening/history and GET /api/screening/{screeningId}       | Done and tested                |
| GET /api/reports                                                      | Done and tested                |
| Passwords and JWT secret moved out of code into environment variables | Done                           |
| Push backend to the shared GitHub repo                                | Pending                        |
| Connect Khushi's frontend to these APIs (api.js)                      | Pending (Khushi)               |
| Connect Karnika's real AI service instead of the mock                 | Pending (needs AI endpoint)    |

## 3. Step-by-step: what was done, in order

### Step 1 - Tools installed and checked

- Java (JDK) - checked with `java -version` (Java 24 installed; the project targets Java 17 and runs fine).
- Git - checked with `git --version`.
- VS Code with the Microsoft **Extension Pack for Java**.
- MySQL Server and MySQL Workbench, Postman or curl.exe for testing.

### Step 2 - Database created

In MySQL Workbench:

```
CREATE DATABASE trueid_db;
```

### Step 3 - Spring Boot project generated

The project was generated from start.spring.io with: Maven, Java, group com.trueid, artifact backend, package com.trueid, dependencies Spring Web, Spring Data JPA, MySQL Driver, Spring Security, Validation (Spring Boot 4.1.1). It was downloaded from the VS Code terminal:

```
curl.exe https://start.spring.io/starter.zip `  
-d type=maven-project -d language=java `  
-d groupId=com.trueid -d artifactId=backend -d name=backend `  
-d packageName=com.trueid -d javaVersion=17 `
```

```
-d dependencies=web,data-jpa,mysql,security,validation `
-o backend.zip
Expand-Archive backend.zip -DestinationPath backend
```

The backend folder (the one containing pom.xml) was then opened in VS Code.

## Step 4 - Configuration and first controller

application.properties was configured for MySQL, then HealthController and a first SecurityConfig were added. The app was started with:

```
.\mvnw.cmd spring-boot:run
```

Opening <http://localhost:8080/api/health> returned the JSON status message, which confirmed that Spring Boot, MySQL and the controller layer work.

## Step 5 - Authentication

JWT libraries were added to pom.xml and application.properties. Files created: User, UserRepository, JwtUtil, AuthService, AuthController, then JwtFilter and the final SecurityConfig. Signup, login and profile were tested with curl.exe.

## Step 6 - Screening

Files created: RiskIndicator, Screening, ScreeningRepository, AIService (mock), ScreeningService, ScreeningController. Upload size limits were added to application.properties. Tested with a normal image (VERIFIED) and an image named with the word 'fake' (REJECTED), then history and get-by-id.

## Step 7 - Reports

ReportService and ReportController were added. The test returned correct totals, verification rate and average confidence.

## Step 8 - Secrets removed from code

The MySQL password and JWT secret now come from environment variables, so they never go to GitHub. A .env.example file with placeholders documents the variables.

# 4. Problems faced and how they were fixed

| Problem                                                                                   | Cause                                                                               | Fix                                                                                                                           |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| pom.xml showed errors: 'zip file is empty' for maven-resources-plugin                     | A Maven plugin download was interrupted and left a corrupt file in the local cache  | Deleted the plugin folder under .m2 and ran mvnw.cmd -U compile                                                               |
| 'Failed to delete target\classes' during clean                                            | OneDrive and the Java extension were locking files in the project folder            | Ran without 'clean'. Keeping the project outside OneDrive (for example C:\dev) avoids this                                    |
| /api/health returned a 404 Whitelabel page                                                | The controller file was misnamed (a comma instead of a dot), so it was not compiled | Renamed it to HealthController.java                                                                                           |
| Compile error: 'class JwtUtil is public, should be declared in a file named JwtUtil.java' | The file name had different capital letters (JWTUtil.java)                          | Renamed the file so it matches the class name exactly. The same happened with JwtFilter                                       |
| App failed to start: 'Port 8080 was already in use'                                       | An earlier server instance was still running                                        | netstat -ano   findstr :8080 to find the PID, then taskkill /PID number /F (no angle brackets), or Ctrl+C in the old terminal |
| curl: option -F: is badly used here                                                       | The -F value was missing 'document=@' before the file path                          | Use -F "document=@C:\path\file.png"                                                                                           |

| Problem                                                    | Cause                                                                         | Fix                                                        |
|------------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------|
| curl: (26) Failed to open/read local data from file        | The image file did not exist at that path (often a hidden .png.png extension) | Checked with Test-Path and copied the image with Copy-Item |
| Unexpected 'Progress (4): ...' text in the server terminal | VS Code's Java extension downloaded JWT libraries in the background           | Not an error; ignored                                      |

## 5. How to run the backend

- MySQL must be running and the database trueid\_db must exist. Tables are created automatically by Hibernate on first start.
- Open the backend folder (the one with pom.xml) in a terminal.
- Set the environment variables in that terminal (use your real MySQL password and any long random string of at least 32 characters):

```
$env:DB_PASSWORD="your_mysql_password"
$env:JWT_SECRET="any_long_random_string_32_chars_or_more"
.\mvnw.cmd spring-boot:run
```

Wait about 30 seconds until the log shows Started BackendApplication. Keep that terminal open and use a second terminal for tests. Always stop the old server with Ctrl+C before starting a new one after code changes. The variables only live in the terminal where they were set, so repeat them in every new terminal.

Optional variable: DB\_USERNAME (default is root). Changing the JWT secret invalidates old tokens; users must log in again but no data is lost.

### Files that must NOT go to GitHub

- Any file containing a real password or secret, including a real .env file.
- backend/target/ (build output).
- backend/uploads/ (uploaded identity documents). Add uploads/ and .env to .gitignore.
- .env.example with placeholders IS meant to go to GitHub.

```
# .env.example
DB_USERNAME=root
DB_PASSWORD=your_mysql_password
JWT_SECRET=any_long_random_string_32_chars_or_more
```

## 6. Backend code, file by file

All Java files live under backend/src/main/java/com/trueid/. File names must match the class names exactly, including capital letters.

```
backend/
  pom.xml
  src/main/resources/application.properties
  src/main/java/com/trueid/
    BackendApplication.java
    controller/ HealthController, AuthController, ScreeningController, ReportController
    service/    AuthService, ScreeningService, AIService (mock), ReportService
    model/      User, Screening, RiskIndicator
    repository/ UserRepository, ScreeningRepository
    security/    SecurityConfig, JwtFilter, JwtUtil
```

### 6.1 Configuration

#### pom.xml (added to the generated dependencies)

The generated project already contains Web, Data JPA, MySQL driver, Security and Validation. These three JWT dependencies were added inside the dependencies section:

```

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>

```

### src/main/resources/application.properties

Database connection, Hibernate behaviour, JWT settings and upload limits. The password and JWT secret come from environment variables. ddl-auto=update makes Hibernate create and update the tables from the entity classes.

```

spring.application.name=backend
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/trueid_db
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

jwt.secret=${JWT_SECRET}
jwt.expiration-ms=86400000

spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=6MB

```

### BackendApplication.java (generated by start.spring.io)

Entry point. Starts Spring Boot and scans all sub-packages of com.trueid.

```

package com.trueid;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {
    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }
}

```

## 6.2 Health check

### controller/HealthController.java

A public endpoint used to confirm that the server is running.

```

package com.trueid.controller;

import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {

    @GetMapping("/api/health")
    public Map<String, String> health() {
        return Map.of("status", "ok", "message", "TrueID backend running");
    }
}

```

## 6.3 Security: JWT and CORS

How it works: after login the backend creates a signed JWT containing the user's email. The frontend sends it in the Authorization header. JwtFilter reads the header on every request, verifies the token with JwtUtil, and marks the request as authenticated. SecurityConfig decides which URLs are public and which need a valid token, allows the frontend origin (http://localhost:5173) through CORS, and provides the BCrypt password encoder.

### security/JwtUtil.java

Creates and reads tokens. The secret and expiry (24 hours) come from application.properties.

```
package com.trueid.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class JwtUtil {

    private final SecretKey key;
    private final long expirationMs;

    public JwtUtil(@Value("${jwt.secret}") String secret,
                  @Value("${jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMs = expirationMs;
    }

    public String generateToken(String email) {
        return Jwts.builder()
            .subject(email)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(key)
            .compact();
    }

    public String extractEmail(String token) {
        return Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload()
            .getSubject();
    }
}
```

### security/JwtFilter.java

Runs once per request. If a valid Bearer token is present, the user is marked as logged in; otherwise the request stays unauthenticated and protected URLs answer 401.

```
package com.trueid.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.List;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class JwtFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;

    public JwtFilter(JwtUtil jwtUtil) {

```

```

        this.jwtUtil = jwtUtil;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {
            try {
                String email = jwtUtil.extractEmail(header.substring(7));
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(email, null, List.of());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (Exception e) {
                // Invalid or expired token: request stays unauthenticated
            }
        }

        chain.doFilter(request, response);
    }
}

```

### security/SecurityConfig.java

Public URLs: /api/health, /api/auth/signup, /api/auth/login. Everything else needs a valid token. Sessions are stateless. Unauthenticated access returns 401 with a JSON error.

```

package com.trueid.security;

import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
public class SecurityConfig {

    private final JwtFilter jwtFilter;

    public SecurityConfig(JwtFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:5173"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.cors(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint((req, res, ex) -> {

```

```

        res.setStatus(401);
        res.setContentType("application/json");
        res.getWriter().write("{\"error\":\"Unauthorized\"}");
    })
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
        .requestMatchers("/api/health", "/api/auth/signup", "/api/auth/login").permitAll()
        .anyRequest().authenticated())
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
}

```

## 6.4 Users and authentication

How it works: signup validates the input, lowercases the email, rejects duplicates, hashes the password with BCrypt and saves the user. Login finds the user by email, compares the password with the stored hash, and returns a token plus the user data. Profile endpoints read the logged-in user's email from the token. The plain password is never stored.

### model/User.java

JPA entity mapped to the MySQL table 'users'. Email is unique.

```

package com.trueid.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String passwordHash;

    private String organization;

    private String role = "Screening Officer";

    private LocalDateTime createdAt = LocalDateTime.now();

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
    public String getOrganization() { return organization; }
    public void setOrganization(String organization) { this.organization = organization; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}

```

### repository/UserRepository.java

Spring Data generates the SQL from the method names.

```

package com.trueid.repository;

import com.trueid.model.User;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

```

```

        Optional<User> findByEmail(String email);
        boolean existsByEmail(String email);
    }

```

### service/AuthService.java

Business logic for signup, login, and profile get/update. Errors are thrown as `IllegalArgumentException` and turned into JSON errors by the controller.

```

package com.trueid.service;

import com.trueid.model.User;
import com.trueid.repository.UserRepository;
import com.trueid.security.JwtUtil;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    public AuthService(UserRepository userRepository,
                      PasswordEncoder passwordEncoder,
                      JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtil = jwtUtil;
    }

    public Map<String, Object> signup(String name, String email,
                                     String organization, String password) {
        if (name == null || name.isBlank() || email == null || email.isBlank()
            || password == null || password.length() < 6) {
            throw new IllegalArgumentException(
                "Name, email and a password of at least 6 characters are required");
        }

        String cleanEmail = email.trim().toLowerCase();

        if (userRepository.existsByEmail(cleanEmail)) {
            throw new IllegalArgumentException("Email already registered");
        }

        User user = new User();
        user.setName(name.trim());
        user.setEmail(cleanEmail);
        user.setOrganization(organization);
        user.setPasswordHash(passwordEncoder.encode(password));
        userRepository.save(user);

        return buildResponse(user);
    }

    public Map<String, Object> login(String email, String password) {
        if (email == null || password == null) {
            throw new IllegalArgumentException("Email and password are required");
        }

        User user = userRepository.findByEmail(email.trim().toLowerCase())
            .orElseThrow(() -> new IllegalArgumentException("Invalid email or password"));

        if (!passwordEncoder.matches(password, user.getPasswordHash())) {
            throw new IllegalArgumentException("Invalid email or password");
        }

        return buildResponse(user);
    }

    public Map<String, Object> getProfile(String email) {
        return userData(findUser(email));
    }

    public Map<String, Object> updateProfile(String email, String name, String organization) {
        User user = findUser(email);

```

```

        if (name != null && !name.isBlank()) {
            user.setName(name.trim());
        }
        if (organization != null) {
            user.setOrganization(organization.trim());
        }

        userRepository.save(user);
        return userData(user);
    }

    private User findUser(String email) {
        return userRepository.findByEmail(email)
            .orElseThrow(() -> new IllegalArgumentException("User not found"));
    }

    private Map<String, Object> userData(User user) {
        Map<String, Object> userData = new LinkedHashMap<>();
        userData.put("id", user.getId());
        userData.put("name", user.getName());
        userData.put("email", user.getEmail());
        userData.put("organization", user.getOrganization());
        userData.put("role", user.getRole());
        return userData;
    }

    private Map<String, Object> buildResponse(User user) {
        Map<String, Object> response = new LinkedHashMap<>();
        response.put("token", jwtUtil.generateToken(user.getEmail()));
        response.put("user", userData(user));
        return response;
    }
}

```

### controller/AuthController.java

Exposes /api/auth/signup, /login, and /profile (GET and PUT). Wrong login returns 401; validation errors return 400; unknown user returns 404.

```

package com.trueid.controller;

import com.trueid.service.AuthService;
import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    record SignupRequest(String name, String email, String organization, String password) {}

    record LoginRequest(String email, String password) {}

    record ProfileRequest(String name, String organization) {}

    @PostMapping("/signup")
    public ResponseEntity<?> signup(@RequestBody SignupRequest req) {
        try {
            return ResponseEntity.ok(authService.signup(
                req.name(), req.email(), req.organization(), req.password()));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {

```

```

    try {
        return ResponseEntity.ok(authService.login(req.email(), req.password()));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(401).body(Map.of("error", e.getMessage()));
    }
}

@GetMapping("/profile")
public ResponseEntity<?> getProfile(Authentication authentication) {
    try {
        return ResponseEntity.ok(authService.getProfile(authentication.getName()));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
    }
}

@PutMapping("/profile")
public ResponseEntity<?> updateProfile(Authentication authentication,
                                      @RequestBody ProfileRequest req) {
    try {
        return ResponseEntity.ok(authService.updateProfile(
            authentication.getName(), req.name(), req.organization()));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
    }
}
}

```

## 6.5 Screening

How it works: the frontend sends a multipart request with documentType, purpose and the file. ScreeningService validates the input (file present; JPG, PNG or PDF only; max 5 MB by config), saves the file in the backend/uploads folder with a random name, asks AIService for a result, saves everything in MySQL, creates the id SCR-(1000 + database id), and returns the result JSON. History and get-by-id only return the logged-in user's own screenings. Tables created: screenings and screening\_risk\_indicators.

### model/RiskIndicator.java

A small embedded object (code, severity, message), stored in its own table with the screening.

```

package com.trueid.model;

import jakarta.persistence.Embeddable;

@Embeddable
public class RiskIndicator {

    private String code;
    private String severity;
    private String message;

    public RiskIndicator() {}

    public RiskIndicator(String code, String severity, String message) {
        this.code = code;
        this.severity = severity;
        this.message = message;
    }

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getSeverity() { return severity; }
    public void setSeverity(String severity) { this.severity = severity; }
    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }
}

```

### model/Screening.java

One screening record: owner, document details, saved file path, and the AI result fields.

```

package com.trueid.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

```

```

@Entity
@Table(name = "screenings")
public class Screening {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String screeningId;

    @ManyToOne(optional = false)
    private User user;

    private String documentType;
    private String purpose;
    private String originalFileName;
    private String filePath;

    private String status;
    private String risk;
    private Integer confidence;
    private Boolean nameMatch;
    private Boolean documentValid;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "screening_risk_indicators",
        joinColumns = @JoinColumn(name = "screening_id"))
    private List<RiskIndicator> riskIndicators = new ArrayList<>();

    private LocalDateTime createdAt = LocalDateTime.now();

    public Long getId() { return id; }
    public String getScreeningId() { return screeningId; }
    public void setScreeningId(String screeningId) { this.screeningId = screeningId; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
    public String getDocumentType() { return documentType; }
    public void setDocumentType(String documentType) { this.documentType = documentType; }
    public String getPurpose() { return purpose; }
    public void setPurpose(String purpose) { this.purpose = purpose; }
    public String getOriginalFileName() { return originalFileName; }
    public void setOriginalFileName(String originalFileName) { this.originalFileName = originalFileName; }
    public String getFilePath() { return filePath; }
    public void setFilePath(String filePath) { this.filePath = filePath; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public String getRisk() { return risk; }
    public void setRisk(String risk) { this.risk = risk; }
    public Integer getConfidence() { return confidence; }
    public void setConfidence(Integer confidence) { this.confidence = confidence; }
    public Boolean getNameMatch() { return nameMatch; }
    public void setNameMatch(Boolean nameMatch) { this.nameMatch = nameMatch; }
    public Boolean getDocumentValid() { return documentValid; }
    public void setDocumentValid(Boolean documentValid) { this.documentValid = documentValid; }
    public List<RiskIndicator> getRiskIndicators() { return riskIndicators; }
    public void setRiskIndicators(List<RiskIndicator> riskIndicators) { this.riskIndicators = riskIndicators; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}

```

### repository/ScreeningRepository.java

```

package com.trueid.repository;

import com.trueid.model.Screening;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ScreeningRepository extends JpaRepository<Screening, Long> {
    List<Screening> findByIdOrderByCreatedAtDesc(Long userId);
    Optional<Screening> findByScreeningIdAndUserId(String screeningId, Long userId);
}

```

### service/AIService.java (MOCK - to be replaced by the real AI call)

This is the only file that changes when Karnika's AI service is ready. It currently returns a fixed VERIFIED result, or a REJECTED result when the uploaded file name contains the word 'fake' (used for testing the

rejected screen). The AiResult shape matches the contract in section 10.

```
package com.trueid.service;

import com.trueid.model.RiskIndicator;
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class AIService {

    public record AiResult(String status, String risk, int confidence,
                          boolean nameMatch, boolean documentValid,
                          List<RiskIndicator> riskIndicators) {}

    // MOCK: replace this with a call to Karnika's POST /ai/analyze later.
    // For testing, a file name containing "fake" returns a rejected result.
    public AiResult analyze(String originalFileName, String documentType) {
        String name = originalFileName == null ? "" : originalFileName.toLowerCase();

        if (name.contains("fake")) {
            return new AiResult("REJECTED", "HIGH", 38, false, false, List.of(
                new RiskIndicator("IMAGE_TAMPERING", "HIGH", "Potential alteration detected"),
                new RiskIndicator("NAME_MISMATCH", "MEDIUM", "Name does not match the account")));
        }

        return new AiResult("VERIFIED", "LOW", 96, true, true, List.of());
    }
}
```

### service/ScreeningService.java

```
package com.trueid.service;

import com.trueid.model.RiskIndicator;
import com.trueid.model.Screening;
import com.trueid.model.User;
import com.trueid.repository.ScreeningRepository;
import com.trueid.repository.UserRepository;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.UUID;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

@Service
public class ScreeningService {

    private static final Set<String> ALLOWED_TYPES =
        Set.of("image/jpeg", "image/png", "application/pdf");

    private final ScreeningRepository screeningRepository;
    private final UserRepository userRepository;
    private final AIService aiService;

    public ScreeningService(ScreeningRepository screeningRepository,
                          UserRepository userRepository,
                          AIService aiService) {
        this.screeningRepository = screeningRepository;
        this.userRepository = userRepository;
        this.aiService = aiService;
    }

    public Map<String, Object> create(String email, String documentType,
                                     String purpose, MultipartFile file) throws IOException {
        if (documentType == null || documentType.isBlank()) {
            throw new IllegalArgumentException("documentType is required");
        }
        if (file == null || file.isEmpty()) {
            throw new IllegalArgumentException("A document file is required");
        }
        if (file.getContentType() == null || !ALLOWED_TYPES.contains(file.getContentType())) {
            throw new IllegalArgumentException("Only JPG, PNG or PDF files are allowed");
        }
    }
}
```

```

    }

    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("User not found"));

    // Save the file with a random name
    Path uploadDir = Paths.get("uploads");
    Files.createDirectories(uploadDir);
    String original = file.getOriginalFilename() == null ? "document" : file.getOriginalFilename();
    String extension = original.contains(".")
        ? original.substring(original.lastIndexOf('.')) : "";
    Path target = uploadDir.resolve(UUID.randomUUID() + extension);
    Files.copy(file.getInputStream(), target);

    // Run AI analysis (mock for now)
    AIService.AiResult result = aiService.analyze(original, documentType);

    Screening screening = new Screening();
    screening.setUser(user);
    screening.setDocumentType(documentType);
    screening.setPurpose(purpose);
    screening.setOriginalFileName(original);
    screening.setFilePath(target.toString());
    screening.setStatus(result.status());
    screening.setRisk(result.risk());
    screening.setConfidence(result.confidence());
    screening.setNameMatch(result.nameMatch());
    screening.setDocumentValid(result.documentValid());
    screening.setRiskIndicators(new ArrayList<>(result.riskIndicators()));
    screening = screeningRepository.save(screening);

    screening.setScreeningId("SCR-" + (1000 + screening.getId()));
    screening = screeningRepository.save(screening);

    return toMap(screening);
}

public List<Map<String, Object>> history(String email) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("User not found"));
    List<Map<String, Object>> list = new ArrayList<>();
    for (Screening s : screeningRepository.findByUserIdOrderByCreatedAtDesc(user.getId())) {
        list.add(toMap(s));
    }
    return list;
}

public Map<String, Object> getOne(String email, String screeningId) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("User not found"));
    Screening s = screeningRepository.findByScreeningIdAndUserId(screeningId, user.getId())
        .orElseThrow(() -> new IllegalArgumentException("Screening not found"));
    return toMap(s);
}

private Map<String, Object> toMap(Screening s) {
    List<Map<String, Object>> indicators = new ArrayList<>();
    for (RiskIndicator r : s.getRiskIndicators()) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("code", r.getCode());
        m.put("severity", r.getSeverity());
        m.put("message", r.getMessage());
        indicators.add(m);
    }

    Map<String, Object> map = new LinkedHashMap<>();
    map.put("screeningId", s.getScreeningId());
    map.put("documentType", s.getDocumentType());
    map.put("purpose", s.getPurpose());
    map.put("status", s.getStatus());
    map.put("risk", s.getRisk());
    map.put("confidence", s.getConfidence());
    map.put("nameMatch", s.getNameMatch());
    map.put("documentValid", s.getDocumentValid());
    map.put("riskIndicators", indicators);
    map.put("createdAt", s.getCreatedAt().toString());
    return map;
}
}

```

## controller/ScreeningController.java

```
package com.trueid.controller;

import com.trueid.service.ScreeningService;
import java.io.IOException;
import java.util.Map;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

@RestController
@RequestMapping("/api/screening")
public class ScreeningController {

    private final ScreeningService screeningService;

    public ScreeningController(ScreeningService screeningService) {
        this.screeningService = screeningService;
    }

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<?> create(Authentication authentication,
                                    @RequestParam String documentType,
                                    @RequestParam(required = false) String purpose,
                                    @RequestParam("document") MultipartFile document) {
        try {
            return ResponseEntity.ok(screeningService.create(
                authentication.getName(), documentType, purpose, document));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        } catch (IOException e) {
            return ResponseEntity.internalServerError()
                .body(Map.of("error", "Could not save the uploaded file"));
        }
    }

    @GetMapping("/history")
    public ResponseEntity<?> history(Authentication authentication) {
        return ResponseEntity.ok(screeningService.history(authentication.getName()));
    }

    @GetMapping("/{screeningId}")
    public ResponseEntity<?> getOne(Authentication authentication,
                                    @PathVariable String screeningId) {
        try {
            return ResponseEntity.ok(screeningService.getOne(
                authentication.getName(), screeningId));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
        }
    }
}
```

## 6.6 Reports

How it works: loads all screenings of the logged-in user and counts them in Java: totals per status and risk level, verification rate (verified / total, as a percentage), average confidence, counts per document type, and the 5 most recent screenings.

## service/ReportService.java

```
package com.trueid.service;

import com.trueid.model.Screening;
import com.trueid.model.User;
import com.trueid.repository.ScreeningRepository;
import com.trueid.repository.UserRepository;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
```

```

import org.springframework.stereotype.Service;

@Service
public class ReportService {

    private final ScreeningRepository screeningRepository;
    private final UserRepository userRepository;

    public ReportService(ScreeningRepository screeningRepository,
                        UserRepository userRepository) {
        this.screeningRepository = screeningRepository;
        this.userRepository = userRepository;
    }

    public Map<String, Object> summary(String email) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new IllegalArgumentException("User not found"));

        List<Screening> list =
            screeningRepository.findByUserIdOrderByCreatedAtDesc(user.getId());

        int total = list.size();
        int verified = 0, review = 0, rejected = 0;
        int low = 0, medium = 0, high = 0;
        int confidenceSum = 0, confidenceCount = 0;
        Map<String, Integer> byDocumentType = new LinkedHashMap<>();

        for (Screening s : list) {
            if ("VERIFIED".equals(s.getStatus())) verified++;
            else if ("REVIEW".equals(s.getStatus())) review++;
            else if ("REJECTED".equals(s.getStatus())) rejected++;

            if ("LOW".equals(s.getRisk())) low++;
            else if ("MEDIUM".equals(s.getRisk())) medium++;
            else if ("HIGH".equals(s.getRisk())) high++;

            if (s.getConfidence() != null) {
                confidenceSum += s.getConfidence();
                confidenceCount++;
            }

            String type = s.getDocumentType() == null ? "Unknown" : s.getDocumentType();
            byDocumentType.merge(type, 1, Integer::sum);
        }

        Map<String, Object> riskLevels = new LinkedHashMap<>();
        riskLevels.put("low", low);
        riskLevels.put("medium", medium);
        riskLevels.put("high", high);

        List<Map<String, Object>> recent = new ArrayList<>();
        for (Screening s : list.stream().limit(5).toList()) {
            Map<String, Object> m = new LinkedHashMap<>();
            m.put("screeningId", s.getScreeningId());
            m.put("documentType", s.getDocumentType());
            m.put("status", s.getStatus());
            m.put("risk", s.getRisk());
            m.put("confidence", s.getConfidence());
            m.put("createdAt", s.getCreatedAt().toString());
            recent.add(m);
        }

        Map<String, Object> report = new LinkedHashMap<>();
        report.put("totalScreenings", total);
        report.put("verified", verified);
        report.put("review", review);
        report.put("rejected", rejected);
        report.put("verificationRate", total == 0 ? 0 : Math.round(verified * 100.0 / total));
        report.put("averageConfidence",
            confidenceCount == 0 ? 0 : Math.round((double) confidenceSum / confidenceCount));
        report.put("riskLevels", riskLevels);
        report.put("byDocumentType", byDocumentType);
        report.put("recentScreenings", recent);
        return report;
    }
}

```

### controller/ReportController.java

```

package com.trueid.controller;

```

```

import com.trueid.service.ReportService;
import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/reports")
public class ReportController {

    private final ReportService reportService;

    public ReportController(ReportService reportService) {
        this.reportService = reportService;
    }

    @GetMapping
    public ResponseEntity<?> summary(Authentication authentication) {
        try {
            return ResponseEntity.ok(reportService.summary(authentication.getName()));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
        }
    }
}

```

## 7. How a request flows through the backend

### Login and a protected request

1. POST /api/auth/login {email, password} (public)  
AuthController -> AuthService -> UserRepository (find by email) -> BCrypt compare  
-> JwtUtil.generateToken(email) -> returns { token, user }
2. Frontend stores the token (for example in localStorage)
3. GET /api/auth/profile with header Authorization: Bearer TOKEN  
JwtFilter reads the header -> JwtUtil.extractEmail -> request marked as logged in  
SecurityConfig allows it -> AuthController uses authentication.getName() (the email)  
-> returns the user's data  
Without a valid token: 401 {"error": "Unauthorized"}

### Screening upload

```

POST /api/screening (multipart: documentType, purpose, document) + Bearer token
-> ScreeningController
-> ScreeningService.create:
    validate input and file type
    save file to backend/uploads/RANDOM-NAME.ext
    AIService.analyze(...) <- MOCK now, Karnika's FastAPI later
    save Screening + RiskIndicators in MySQL
    screeningId = "SCR-" + (1000 + database id)
-> returns result JSON (status, risk, confidence, nameMatch, documentValid, riskIndicators)

```

## 8. API reference (the contract)

Base URL (local): <http://localhost:8080>. Allowed frontend origin: <http://localhost:5173>. All bodies are JSON except the screening upload (multipart/form-data). Every endpoint except health, signup and login needs the header Authorization: Bearer TOKEN. Tokens last 24 hours. Errors always look like {"error": "message"}.

| Method and path       | Auth | Purpose                                |
|-----------------------|------|----------------------------------------|
| GET /api/health       | No   | Server status                          |
| POST /api/auth/signup | No   | Create account; returns token and user |
| POST /api/auth/login  | No   | Log in; returns token and user         |
| GET /api/auth/profile | Yes  | Current user                           |



```

"documentType": "Aadhaar",
"purpose": "Identity Verification",
"status": "REJECTED",
"risk": "HIGH",
"confidence": 38,
"nameMatch": false,
"documentValid": false,
"riskIndicators": [
  { "code": "IMAGE_TAMPERING", "severity": "HIGH", "message": "Potential alteration detected" },
  { "code": "NAME_MISMATCH", "severity": "MEDIUM", "message": "Name does not match the account" }
],
"createdAt": "2026-10-03T14:51:09.435438"
}

```

status: VERIFIED | REVIEW | REJECTED      risk and severity: LOW | MEDIUM | HIGH  
confidence: number 0 to 100

Errors: 400 {"error":"Only JPG, PNG or PDF files are allowed"}  
400 {"error":"A document file is required"}  
400 {"error":"documentType is required"}

## History and single screening

GET /api/screening/history   -> array of screening objects (same shape as above), newest first;  
empty history returns []  
GET /api/screening/SCR-1001   -> one screening object  
404 {"error":"Screening not found"} (users only see their own)

## Reports

```

GET /api/reports
{
  "totalScreenings": 2,
  "verified": 1,
  "review": 0,
  "rejected": 1,
  "verificationRate": 50,
  "averageConfidence": 67,
  "riskLevels": { "low": 1, "medium": 0, "high": 1 },
  "byDocumentType": { "Aadhaar": 2 },
  "recentScreenings": [
    { "screeningId": "SCR-1002", "documentType": "Aadhaar", "status": "REJECTED",
      "risk": "HIGH", "confidence": 38, "createdAt": "2026-10-03T14:51:09.435438" },
    { "screeningId": "SCR-1001", "documentType": "Aadhaar", "status": "VERIFIED",
      "risk": "LOW", "confidence": 96, "createdAt": "2026-10-03T14:36:23.483015" }
  ]
}

```

verificationRate is a percentage; recentScreenings has up to 5 items.

## 9. How to test (PowerShell, curl.exe)

Run the server in one terminal and these commands in a second terminal. Replace the image path with a real .jpg or .png on your computer. The -F value must start with document=@

```

# Get a token (first create the account with POST /api/auth/signup)
$login = @{ email="harshita@test.com"; password="test1234" } | ConvertTo-Json
$res = $login | curl.exe -s -X POST http://localhost:8080/api/auth/login `
  -H "Content-Type: application/json" --data-binary "$login" | ConvertFrom-Json
$token = $res.token

# Without token: should return 401
curl.exe -i http://localhost:8080/api/auth/profile

# With token: should return 200 and the user
curl.exe -i http://localhost:8080/api/auth/profile -H "Authorization: Bearer $token"

# Update profile
$upd = @{ name="Harshita K"; organization="TrueID Labs" } | ConvertTo-Json
$upd | curl.exe -i -X PUT http://localhost:8080/api/auth/profile `
  -H "Content-Type: application/json" -H "Authorization: Bearer $token" --data-binary "$upd"

# Upload a document (a file name containing "fake" returns the REJECTED mock result)
curl.exe -i -X POST http://localhost:8080/api/screening `
  -H "Authorization: Bearer $token" -F "documentType=Aadhaar" `
  -F "purpose=Identity Verification" -F "document=@C:\Users\Dell\Desktop\test.png"

# History, single screening, reports

```

```
curl.exe -i http://localhost:8080/api/screening/history -H "Authorization: Bearer $token"
curl.exe -i http://localhost:8080/api/screening/SCR-1001 -H "Authorization: Bearer $token"
curl.exe -i http://localhost:8080/api/reports -H "Authorization: Bearer $token"
```

In PowerShell never write angle brackets around placeholders such as a process id; `taskkill /PID 12345 /F` is correct.

## 10. Contract between the backend and Karnika's AI service

For Karnika (and anyone writing the AI part). The backend will forward the uploaded document to the AI service and store whatever it returns. The response below is exactly what the frontend result page displays, so the field names, value sets and types must not change. Until the AI service exists, `AIService.java` returns mock data in this same format.

### Proposed AI endpoint (to be confirmed by Karnika)

```
POST /ai/analyze      Content-Type: multipart/form-data
file                 required   JPG, PNG or PDF, max 5 MB
documentType         text       required   Aadhaar | Passport | DL | VoterID
selfie               file       optional   JPG or PNG, for face matching (if supported)
```

Success 200:

```
{
  "status": "VERIFIED",
  "risk": "LOW",
  "confidence": 96,
  "nameMatch": true,
  "documentValid": true,
  "riskIndicators": [
    { "code": "IMAGE_TAMPERING", "severity": "HIGH", "message": "Potential alteration detected" }
  ],
  "extractedData": { "name": "", "dob": "", "idNumber": "" }
}
```

status: VERIFIED | REVIEW | REJECTED  
risk: LOW | MEDIUM | HIGH  
confidence: number 0 to 100  
riskIndicators: array of objects (code, severity LOW|MEDIUM|HIGH, message); [] when none  
extractedData: optional (name, dob, idNumber read by OCR)

Error (4xx or 5xx): { "error": "message" }

### Division of work

| Spring Boot backend (Harshita)                   | FastAPI AI service (Karnika)                      |
|--------------------------------------------------|---------------------------------------------------|
| Authentication and JWT                           | Document type detection                           |
| Database (users, screenings, indicators)         | OCR (name, date of birth, ID number)              |
| File upload and validation of file type and size | Document structure and field validation           |
| Calling the AI service and saving its result     | Tampering and forgery detection                   |
| History and reports                              | Face comparison (document photo vs captured face) |
| Returning the final result to the frontend       | Confidence and risk calculation                   |

The backend does not need its own OCR. The planned `OCRService.java` from the structure file is not needed if OCR stays in the AI service.

### What changes when the real AI is connected

- Only `AIService.java` changes: instead of returning fixed data it sends the file and `documentType` to the AI service URL and maps the JSON response to the same `AiResult`.
- The AI service URL will be a configuration value (for example `http://localhost:8000`).
- If the AI service is down or returns an error, the backend should return a clear JSON error instead of saving a result.

- No frontend change is needed, because the response format stays the same.

## Open questions for the team

- Karnika: what is the AI service base URL, and does it accept a selfie for face matching?
- Name match: what should the document name be compared with? The account name, or a name entered by the user on the screening page?
- Khushi: does CameraScanner capture the document or a face? Which fields do the Result, History and Reports pages need beyond the ones above?

## 11. Notes for connecting the frontend (Khushi)

- The current frontend login is fake (localStorage flags) and the result page shows mock data. Replace them with calls to the endpoints in section 8, one page at a time: api.js, SignIn, SignUp, Profile, NewScreening and DocumentUpload, Result, History, Reports.
- The frontend must run on http://localhost:5173. The backend only allows that origin; any other port causes a CORS error.
- Send the token on protected requests: Authorization: Bearer TOKEN.
- For the upload, use FormData and do NOT set the Content-Type header yourself.
- Images captured by CameraScanner must be converted to a file or Blob and sent in the 'document' field.
- On any 401 from a protected endpoint, clear the stored token and go to the Sign In page.
- The backend runs on the backend developer's computer. Teammates must either run the backend and MySQL locally from GitHub or test together while the backend developer runs it.

```
// Login
const res = await fetch("http://localhost:8080/api/auth/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password }),
});
const data = await res.json();
if (!res.ok) throw new Error(data.error);
localStorage.setItem("trueid_token", data.token);
localStorage.setItem("trueid_user", JSON.stringify(data.user));

// Upload (no manual Content-Type)
const form = new FormData();
form.append("documentType", "Aadhaar");
form.append("purpose", "Identity Verification");
form.append("document", file);
const r = await fetch("http://localhost:8080/api/screening", {
  method: "POST",
  headers: { Authorization: "Bearer " + localStorage.getItem("trueid_token") },
  body: form,
});
const result = await r.json();
```

## 12. Next steps

| # | Task                                                                                                                | Owner    |
|---|---------------------------------------------------------------------------------------------------------------------|----------|
| 1 | Push the backend and docs to the shared TrueID-AI GitHub repository (never push passwords, .env, target or uploads) | Harshita |
| 2 | Build frontend/src/services/api.js and connect each page to the real API                                            | Khushi   |
| 3 | Agree the AI contract (section 10) and expose POST /ai/analyze                                                      | Karnika  |
| 4 | Replace the mock AIService with a real call to the AI service                                                       | Harshita |
| 5 | End-to-end test: frontend, backend, AI, database                                                                    | All      |

| # | Task                                                           | Owner              |
|---|----------------------------------------------------------------|--------------------|
| 6 | Write docs/AI_DOCUMENTATION.md and docs/SYSTEM_ARCHITECTURE.md | Karnika / Harshita |

## 13. Security notes

- Passwords are stored only as BCrypt hashes. The plain password is never saved.
- The MySQL password and JWT secret live only in environment variables. Each teammate uses their own values; never share or commit them. If a real password was ever pasted into chats or screenshots, change it.
- Uploaded identity documents are sensitive (for example Aadhaar). Keep backend/uploads out of GitHub, never log document contents, and mention data handling and deletion in the project report.
- Users can only read their own screenings and reports.
- CORS currently allows only `http://localhost:5173`. Update it when the frontend is deployed.
- The temporary generated-password warning in the server log is from Spring Security's default user and is unused, because login is handled by AuthService.